CPSC 375 High-Performance Computing

Lecture 9

Sockets

Basic Concepts

Linux IPC

- Pipes
- Message Queues
- Shared memory
- Semaphores
- Signals
- Sockets

What Are Sockets?

- Local IPC
  - designed for communication between processes running on the same operating system kernel
  - e.g., pipes, shared memory, message queues
- Processes communicate across different physical machines through their sockets
  - Communicate through message passing mechanisms
  - Data is explicitly copied from one process's memory to another's on a different host.
  - Sockets as the conduit for the distributed memory model

Addressing Processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- *Q:* Does IP address of host on which process runs suffice for identifying the process?
- *A:* No, *many* processes can be running on the same host

- *identifier* includes both IP address and port numbers associated with process on host.
- example port numbers:
  - HTTP server: 80
  - mail server: 25
- to send HTTP message to *www.cs.trincoll.edu* web server:
  - IP address: 157.252.16.12
  - port number: 80

Supported Services: TCP

TCP (Transmission Control Protocol) *service:*

- *connection-oriented:* setup required between client and server processes
- *reliable transport* between sending and receiving process
- *flow control:* sender won't overwhelm receiver
- *congestion control:* throttle sender when network overloaded
- *does not provide:* timing, minimum throughput guarantee, security
- e.g., HTTP, FTP, SSH, SMTP

Supported Services: UDP

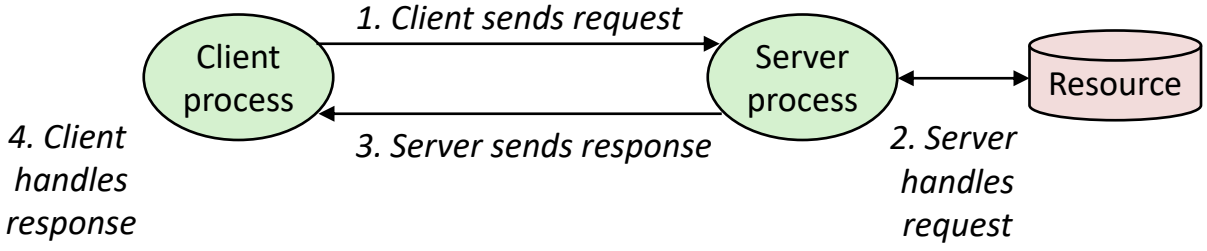
UDP (User Diagram Protocol) *service:*

- *connectionless:* no handshaking before the two processes start to communicate.
- *unreliable data transfer* between sending and receiving process
- *does not provide:* reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup,

Q: Why bother? Why is there a UDP?

- commonly used for real-time applications:
  - audio/video streaming, multiplayer video games, etc.
  - *DNS* for resolving domain names
  - apps that don't require a response from the other end, e.g., IP *broadcasting* or *multicasting*

Client-Server Transaction



Note: clients and servers are processes running on hosts (can be the same or different hosts)

Using Sockets

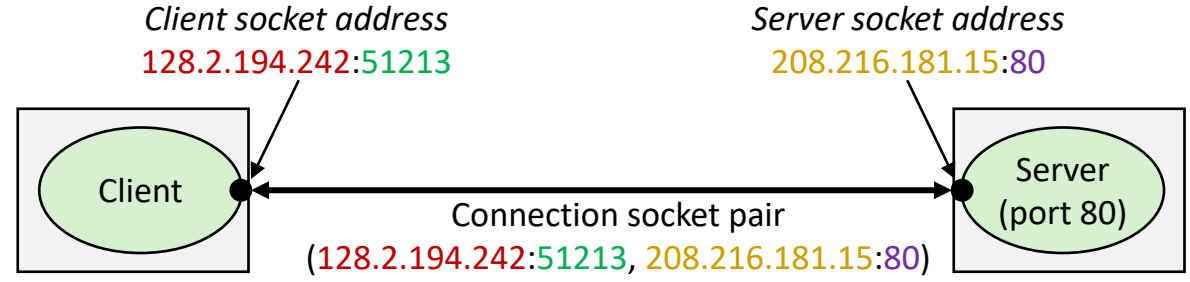
- Clients and servers communicate by sending streams of bytes over *connections*:
  - Point-to-point, full-duplex (2-way communication), and reliable.
- A *socket* is an endpoint of a connection
  - socket address is an `IPaddress:port` pair

Ports

- A *port* is a 16-bit integer that identifies a process:
  - *ephemeral port*: assigned automatically on client when client makes a connection request
  - *well-known port:* associated with some service provided by a server (e.g., port 80 is associated with Web servers)
- A connection is uniquely identified by the socket addresses of its endpoints (*socket pair*)

  `(cliaddr:cliport, servaddr:servport)`

Client-Server Connection Using Sockets



51213 is an ephemeral port allocated by the kernel

80 is a well-known port associated with Web servers